LFU and FIFO Eviction

when recency alone is fooled

Phase 3 videos on YouTube

In one line: LFU evicts the entry with the lowest access count (with decay so old fame fades), which keeps long-term best-sellers and resists scans; FIFO evicts the oldest insertion and is cheap but blind; cache pollution is useless entries pushing out useful ones, and modern caches such as W-TinyLFU combine recency and frequency.

1 Recency gets fooled

A nightly batch job scans a million old products once each. With an LRU cache (lesson 20), every one of them becomes the most recently used entry, and together they push out your truly popular items. By morning the best-sellers are gone from the cache and the hit ratio collapses. Recency alone was fooled. Two other policies help: LFU and FIFO.

2 LFU: least frequently used

Each entry keeps a **counter** of how many times it has been accessed. When the cache is full, evict the entry with the **lowest count**. Items that are popular over a long period, like top products or the home page, keep high counts and stay cached, even while a burst of other keys passes through.

Pure LFU has a problem: **old popularity never fades**. Last month's viral item keeps its huge count forever, and new hot items can't displace it. The fix is **decay**: counters are periodically halved, or decrease over time, so frequency reflects *recent* popularity. Redis's LFU policies (`allkeys-lfu`, `volatile-lfu`) use an 8-bit logarithmic counter per key with a configurable decay time (`lfu-decay-time`, 1 minute by default).

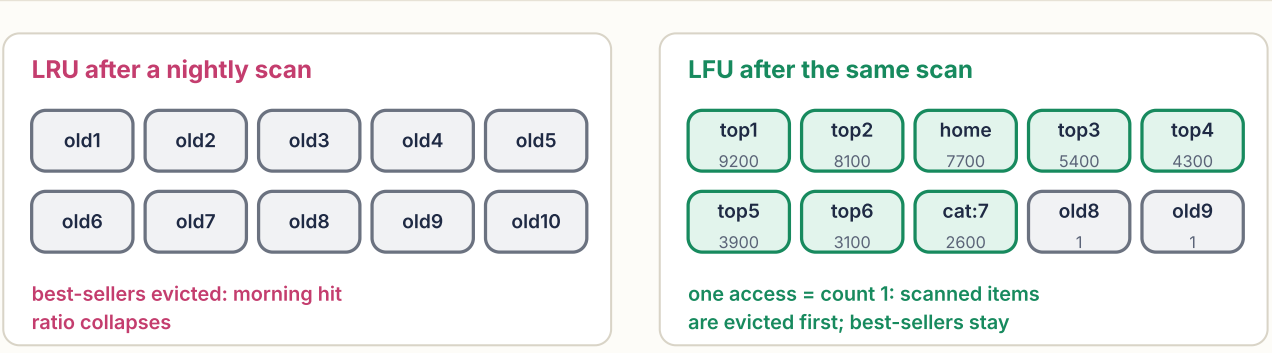


Figure 1. After a scan, LRU holds the scanned items; LFU still holds the best-sellers, because one access gives only a count of 1.

3 FIFO: first in, first out

The simplest policy: evict the entry that was **inserted** earliest, regardless of how it's been used. It needs no bookkeeping on reads, which makes it cheap and easy to scale (a read never has to update shared state). But it may evict a hot item just because it's old. FIFO works when entries have similar value, or when combined with TTLs. Recent research (S3-FIFO) shows a FIFO with a small probation queue can match LRU on real workloads at lower cost.

policy		keeps track of	weakness
LRU	least recently used	recency	fooled by scans
LFU	least frequently used	count (with decay)	slow to adapt to new hits
FIFO	first in, first out	insertion order	evicts hot items for being old
W-TinyLFU	Caffeine	admission by frequency sketch	the best all-rounder

Figure 2. What each eviction policy keeps track of, and where it goes wrong.

4 Cache pollution

Cache pollution is when entries that will never be used again fill the cache and push out useful ones. Scans, crawlers and one-off reports are common causes. LRU is vulnerable, because a single access makes an item look hot. LFU resists pollution, because one access gives a low count.

WORKED EXAMPLE: AN ONLINE STORE'S CATALOGUE

The store's top products are viewed all day, every day. With Redis's `allkeys-lfu` policy, those products build high frequency counts and stay cached, while one-off views from crawlers and long-tail searches get evicted first. The catalogue hit ratio stays around 95% even while a crawler walks through every product page.

5 The modern answer: TinyLFU

TinyLFU uses a compact frequency sketch (a count-min sketch: a few small counter arrays indexed by hashes, a few bytes per entry) to decide whether a new item is worth **admitting** at all: it only lets the item in if it's more popular than the item it would evict. **Window TinyLFU**, used by Java's Caffeine library, adds a small LRU window (about 1% of the cache) so brand-new items get a chance to build up frequency. It beats pure LRU and pure LFU on most real workloads.

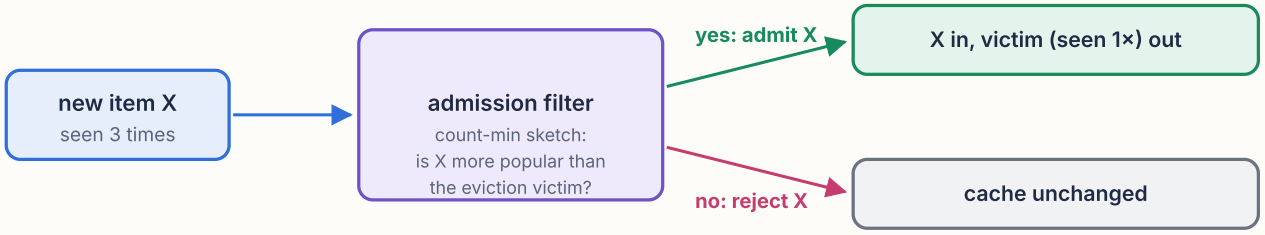


Figure 3. TinyLFU's admission filter: a new item only replaces the eviction victim if it has been seen more often.

COMMON MISTAKES

- LFU without decay: yesterday's viral key is cached forever.
- Assuming LRU everywhere, then losing the hot set to every batch job and crawler.
- Changing the eviction policy without measuring the hit ratio before and after on real traffic.

6 Key terms

LFU: evict the lowest access count

FIFO: evict the earliest inserted

Scan resistance: a one-off scan doesn't flush the hot set

Decay: counts shrink over time so old fame fades

Cache pollution: never-reused entries evicting useful ones

Count-min sketch: compact approximate frequency counter

TinyLFU: admit a new item only if it's more popular

W-TinyLFU: TinyLFU plus a small LRU window (Caffeine)

7 Interview questions

Q1. When does LFU beat LRU? BASIC

When popularity is stable over time, like top products, and when scans or one-off accesses would otherwise push hot items out, because LFU needs repeated use to keep an item.

Q2. What is cache pollution? BASIC

Filling the cache with entries that won't be reused, evicting valuable ones; LFU and TinyLFU resist it better than LRU.

Q3. Why does LFU need decay? BASIC

Without it, items that were popular long ago keep high counts forever and block new hot items; decaying counters make frequency reflect recent popularity.

Q4. Your cache hit ratio drops every night when a reporting job runs. What do you change? SCENARIO

Switch to a scan-resistant policy (allkeys-lfu in Redis, W-TinyLFU in-process), or make the job bypass the cache, and measure the hit ratio before and after.

Q5. When is FIFO a reasonable choice? TRICKY

When entries have similar value or short TTLs, when you need reads with no bookkeeping for scalability, or as part of a design like S3-FIFO that adds a probation queue to filter one-hit items.

Q6. How does TinyLFU decide admission cheaply? TRICKY

It keeps approximate access counts in a count-min sketch (with periodic halving), and compares the newcomer's estimated frequency with the eviction candidate's, admitting only if the newcomer is more popular.

Next: SD-F-008 · Cache Invalidation Strategies backendroadmap.com